MCP Usage (Pilot)

This page explains the minimum setup to run the DataSyncX MCP pilot tools in VS Code Copilot.

Pilot scope:

- Read-only tools only
- Current tools: `mcp_health`, `mcp_list_capabilities`, `mcp_sqlserver_read_probe`, `mcp_mars_read_probe`
- No TaskRun execution, writer operations, or DDL/DML SQL

Architecture Note

This MCP pilot is built on DataSyncX components, not direct SQL driver calls in the MCP layer.

Call chain:

- Copilot MCP call -> `datasyncx.mcp.server:mcp_sqlserver_read_probe`
- MCP tool -> `datasyncx.readers.sqlserver_reader.SqlServerReader`
- Reader -> DataSyncX SQL utility path

Mars probe path:

- Copilot MCP call -> `datasyncx.mcp.server:mcp_mars_read_probe`
- MCP tool -> `datasyncx.readers.mars_reader.MarsReader`
- Reader -> DataSyncX Oracle utility path (`query_mao`)

This keeps SQL access behavior inside DataSyncX abstractions.

Prerequisites

- Python 3.11, 3.12, or 3.13
- `datasyncx` installed from this repository (editable install is recommended for development)
- MCP package available in the same environment

Install in the DataSyncX repo-local venv:

```
python -m pip install -e .
python -m pip install "mcp[cli]>=1.0.0,<2"
```

VS Code MCP Registration

Create or update `.vscode/mcp.json` in your active workspace:

```
{
  "servers": {
    "datasyncx-pilot": {
      "command": "${workspaceFolder}/venv/Scripts/python.exe",
      "args": ["-m", "datasyncx.mcp.server"]
    }
  }
}
```

If your environment is `.venv`, replace `venv` with `.venv`.

Validate In Copilot Chat

Open a new Copilot chat in the same workspace and run:

1. Call `mcp_health`
2. Call `mcp_list_capabilities`
3. Call `mcp_sqlserver_read_probe` with `dry_run=true`
4. Call `mcp_mars_read_probe` with `dry_run=true`

Prefer server-qualified wording in prompts so Copilot routes to the intended MCP server:

- `Call datasyncx-pilot mcp_health`
- `Call datasyncx-pilot mcp_list_capabilities`
- `Call datasyncx-pilot mcp_sqlserver_read_probe` with `dry_run=true`
- `Call datasyncx-pilot mcp_mars_read_probe` with `dry_run=true`

Expected `mcp_health` fields:

- `status = ok`
- `service = datasyncx-mcp`
- `mode = pilot-read-only`

Expected `mcp_list_capabilities` fields:

- `namespace = datasyncx`

- `pilot_scope = read_only`
- `contract_version = v1`

Expected `mcp_sqlserver_read_probe` behavior:

- Accepts only read-only `SELECT` or `WITH` SQL text
- Blocks SQL containing write/DDL keywords
- Normalizes `row_limit` and `timeout_sec` to bounded values
- Supports `dry_run=true` for policy-only validation
- Supports `dry_run=false` for real read execution when:
  - query includes SQL row limiter (`TOP n` or `OFFSET/FETCH`) by default
  - or caller explicitly sets `allow_unbounded=true`
  - `connection_profile` includes `server` and `database` (`username` is optional)
  - when `username` is omitted, DataSyncX uses Windows integrated authentication
  - if `connection_profile.password` is omitted, MCP tries DataSyncX `.env` lookup via `get_config_value(username)`
  - if password is still unresolved after `.env` lookup, tool returns `invalid_connection_profile`
- Redacts sensitive fields from `connection_profile` output
- In `dry_run=true`, returns a `warning` when the same query would be blocked in execute mode (for example missing SQL limiter while `allow_unbounded=false`)

Expected `mcp_mars_read_probe` behavior:

- Accepts only read-only `SELECT` or `WITH` SQL text
- Blocks SQL containing write/DDL keywords
- Requires `site` as a non-empty string
- Normalizes `row_limit` and `timeout_sec` to bounded values
- Supports `dry_run=true` for policy-only validation
- Supports `dry_run=false` for real read execution when:
  - query includes SQL row limiter (`TOP n` or `OFFSET/FETCH`) by default
  - or caller explicitly sets `allow_unbounded=true`
- Optional `connection_profile` supports:
  - `username`
  - `password`
  - `db_account`
  - `factory`
- Redacts sensitive fields from `connection_profile` output
- In `dry_run=true`, returns a `warning` when the same query would be blocked in execute mode

Troubleshooting

`ModuleNotFoundError: No module named datasyncx.mcp`

- Cause: installed package does not yet include local MCP files.
- Fix: run editable install in this repo:

```
python -m pip install -e .
```

`Invalid JSON: EOF while parsing a value` when running `python -m datasyncx.mcp.server`

- Cause: `stdio` MCP server is waiting for JSON-RPC input; plain terminal input is not a client request.
- Fix: validate via VS Code Copilot MCP client, not interactive terminal typing.

`allow_unbounded is not a recognized argument` (or newly added tool args are missing)

- Cause: VS Code is still connected to a stale MCP server process or stale tool schema.
- Fix sequence:

```
Set-Location C:\Users\yujietan\PycharmProjects\datasyncx
python -m pip install -e .
Get-CimInstance Win32_Process | Where-Object { $_.CommandLine -like '*datasyncx.mcp.server*' }
| ForEach-Object { Stop-Process -Id $_.ProcessId -Force }
```

Then in VS Code:

1. Run `Developer: Reload Window`
2. Open a new chat session
3. Call `datasyncx-pilot mcp_health`
4. Retry the MCP tool call with the new argument

Tip:

- If behavior still looks stale, confirm editable install points to this repository path with `pip show datasyncx` and check `Editable project location`.